COURSE TITLE: Software Engineering			
Course Code:	24PCA502	***Course Type***	***MJC***
Teaching Hours / Week (L: T: P)	2:0:2	***Credits***	3
Total Teaching Hours	30:0:30	***CIE + SEE Marks***	50+50=100

Preamble.

This course is designed to provide students with a comprehensive understanding of modern software engineering practices, principles, and tools. It covers the Software Development Life Cycle (SDLC) models, software design principles, and critical concepts such as SOLID and DRY. The course emphasizes hands-on experience with tools like Git, Jira, and CI/CD pipelines, ensuring students gain practical skills in version control, project management, and automation. It also focuses on the importance of clean coding practices, testing, and adherence to standards to produce reliable, maintainable, and scalable software systems.

Course Outcomes

At the end of the course students will be able to…

CO 1	Understand the principles of software engineering, including SDLC models, software requirements, and design principles.
CO 2	Apply SOLID and DRY principles to create high-level and low-level designs, transforming requirements into robust software architectures.
CO 3	Utilize version control systems like Git and project management tools like Jira for efficient team collaboration and project tracking.
CO 4	Develop, implement, and monitor CI/CD pipelines to automate software builds and deployments, ensuring streamlined software delivery.
CO 5	Write unit tests, refactor code, and follow clean coding practices to ensure the quality, reliability, and maintainability of software systems.

CO-PO Mapping

	PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**
CO 1	3	2	1	2	1	1	2	3
CO 2	3	3	3	2	1	-	-	3
CO 3	2	3	3	2	3		2	2
CO 4	2	3	3	3	2	1	1	3
CO 5	3	3	2	3	2	1	2	3

Note: ‘-‘=Not Relevance; ‘1’=Low Relevance; ‘2’ = Medium Relevance; ‘3’=High Relevance

Course content

Theory

Unit 1 Introduction **6 hours**

Introduction to Software Engineering: Definition and Importance of Software Engineering. Software Development Life Cycle (SDLC) Models (Waterfall, Agile, Spiral, etc.). Characteristics of Good Software. Challenges in Software Development. Software Requirement Specification (SRS): Definition and Importance of SRS. Components of SRS: Functional and Non-functional Requirements. SRS Documentation Standards (IEEE Standard 830). Techniques for Requirements Gathering and Analysis. Software Design Principles. Introduction to Design Principles: Abstraction, Modularity, and Reusability

Unit 2 **6 hours**

SOLID Principles: Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. DRY (Don't Repeat Yourself) Principle. High-Level Design (HLD): Objectives, Components. Low-Level Design (LLD): Detailing HLD, Focus on Algorithms and Data Structures. SRS to HLD and LLD Workflow: Converting Requirements into Design.

Unit 3 **6 hours**

Version Control Systems (VCS): Introduction, Importance, and Tools (Git Overview).
Git Essentials: Clone, Commit, Push, Pull, Branch, and Merge. vanced Git Commands: Resolving Merge Conflicts, Git Reset (Soft and Hard). Agile Practices: Introduction to Jira, Creating Backlogs, User Stories, Tasks, and Subtasks.

Unit 4
System Hacking Techniques **6 hours**

CI/CD Overview: Importance in Modern Software Development, Tools (GitHub Actions, Jenkins). Setting Up CI Pipelines: Trigger Automated Builds, Monitor Build Status.
Clean Code Practices: Coding Standards, Refactoring Techniques, Pair Programming, Mock Code Review Sessions.

Unit 5
Hacking Mobile Platforms, Cloud and IoT **6 hours**

Unit Testing: Importance, Tools (xUnit, NUnit), Writing Test Cases. Mocking and Dependency Injection in Tests. Testing Standards and Best Practices: Code Coverage, Regression Testing.

Practical:

S.No	Contents (*Practical)*
1	Design a Software Requirement Specification (SRS) Document for a project.
2	Understand software design principles – SOLID and DRY a Unified Modelling Language (UML). Implement UML diagrams – Use case, class, sequence diagram for a project of your choice.
3	Write high level design (HLD) and low-level design (LLD) for a Software Requirements Specification Document.
4	Use Jira board to learn how to create backlogs, story, tasks and sub tasks and understand the software product tasks in day-to-day lifecycle
5	Setup a github repository for the course project and work on prime git commands – git clone, commit, push, pull, branch, merge.
6	Work on advanced git commands – solve merge conflicts, git reset (soft and hard)
7	Choose a small project in github. Understand and split the work into sub problems. Write user stories with acceptance criteria, tasks and subtasks for the same.
8	Learn and understand CI/CD Pipelines. Set up a CI pipeline using GitHub Actions or Jenkins Trigger automated builds on code commits.
9	Consider and choose a small project code of your choice which does not have coding standards and clean code. Write a clean and maintainable code by refactoring the codebase to adhere to coding standards. Conduct a mock code review session. Apply pair programming.
10	Learn and understand unit tests and their importance in development phase. Implement a unit test using xUnit or NUnit in C# for a project of your choice.

Reference Books;

1. Software Engineering: A Practitioner's Approach by Roger S. Pressman, 8th Edition, McGraw Hill.
2. Clean Code: A Handbook of Agile Software Craftsmanship by Robert C. Martin.
3. Design Patterns: Elements of Reusable Object-Oriented Software by Erich Gamma,
4. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation by Jez Humble and David Farley.
5. Git Pro by Scott Chacon and Ben Straub.
6. Unit Testing Principles, Practices, and Patterns by Vladimir Khorikov.

Course Evaluation System

Assessment System	Assessment Component	Description	Weightage	Marks
Theory Continuous Internal Evaluation (CIE)	CIE-I	Mid Semester Examination (MSE) –I of 1 hour duration for 25 marks	--	50
	CIE-II	Mid Semester Examination (MSE) –II of 1 hour duration conducted for 25 marks	--	
	--	Average of MSE-1 and MSE-II	25	
	CIE-III	Assignments/Quizzes/Case Analysis	10	
	CIE-IV	Publications/Seminar/Quiz etc.	10	
	CIE-V	Attendance*	5	
CIE (Theory)			**50%**	**50M**
Semester End Evaluation (SEE)	**SEE**	**Written Examination conducted for 3 hours duration.**	**50%**	**50 M**
		TOTAL MARKS	**100%**	**100**

***Attendance Marks Allotment:** 90% and above : 5 Marks
85% to 89% : 4 Marks
80% to 84% : 3 Marks
75% to 79% : 2 Marks

Question Paper Pattern:
The SEE question paper comprises 5 Modules/Units. Each Module/Unit will have 2 questions carrying 20 marks each. Each question may have sub questions (a, b & c etc) for a total mark of 20. Students need to answer 5 questions selecting one FULL question from each module.

.